

Counters

Ripple & synchronous · up, down, up/down · mod-N · decade · ring · Johnson · frequency division

QUICK TRICK

Four lines to learn first: n flip-flops give at most 2^n states. Mod-N needs $\lceil \log_2 N \rceil$ flip-flops. A mod-N counter divides the clock frequency by **N**. Synchronous counter: $T_1 = \text{AND of all the lower bits}$.

15.1 What is a counter?

● LEVEL 1 · BEGINNER

What is it? A counter is a circuit that **counts clock pulses**. Its flip-flops show the count in binary. After the last count it goes back to 0 and starts again.

Why do we need it? Timers, clocks, memory addresses, and dividing a fast clock into slow ones.

Analogy. A car odometer. Each wheel counts 0–9 and, when it passes 9, it moves the next wheel by one. A binary counter has wheels that count only 0 and 1.

What does “number of states = 2^n ” mean?

Each flip-flop can be 0 or 1. With n flip-flops there are $2 \times 2 \times \dots \times 2 = 2^n$ different patterns. A 3-bit counter can show 000, 001, 010, 011, 100, 101, 110, 111, which is **8 patterns** (8 states). The counting goes 0 to 7, then back to 0. The **modulus** is 8, so it is called a **mod-8** counter.

Flip-flops (n)	States (mod)	Counts from ... to
2	4	0 to 3
3	8	0 to 7
4	16	0 to 15
8	256	0 to 255

QUICK TRICK

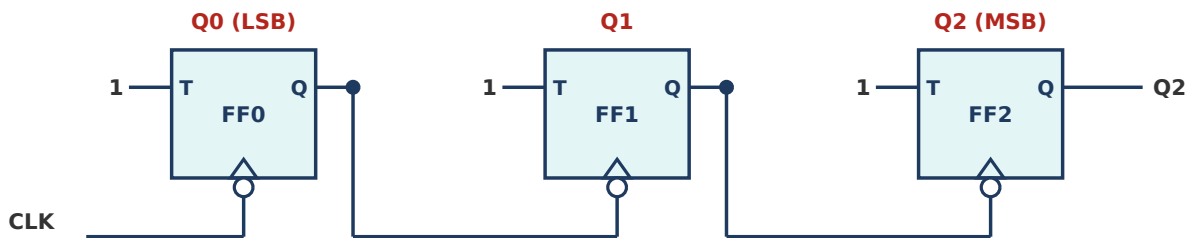
Counting-pulses trick: a mod-N counter that starts at 0 shows **(number of pulses) mod N**. Example: a 4-bit counter after 37 pulses shows $37 \bmod 16 = 5 = \mathbf{0101}$.

	Asynchronous (ripple)	Synchronous
Clock	Only the first flip-flop gets the clock. The others are clocked by the previous flip-flop.	All flip-flops get the same clock .
Speed	Slow (delays add up)	Fast
Circuit	Very simple	Needs extra gates
Glitches in decoding	Yes (flip-flops do not change at the same moment)	Fewer

15.2 Asynchronous (ripple) counter

● LEVEL 2 • EXAM

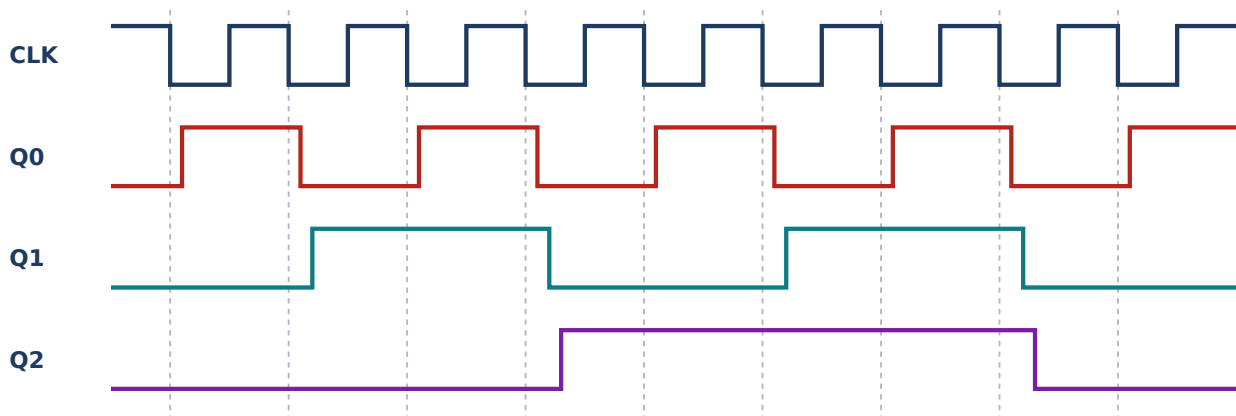
Idea. Use flip-flops that **toggle** (T flip-flops, or JK flip-flops with $J = K = 1$). The clock goes only to the first flip-flop. Each flip-flop **toggles when the one before it falls from 1 to 0**. Like falling dominoes, the change **ripples** through.



This is a **3-bit up counter** built with **negative-edge-triggered** T flip-flops (T tied to 1). Q0 clocks FF1 and Q1 clocks FF2. Q0 is the least significant bit.

Pulse	Q2	Q1	Q0	Decimal
0	0	0	0	0
1	0	0	1	1
2	0	1	0	2
3	0	1	1	3
4	1	0	0	4
5	1	0	1	5
6	1	1	0	6
7	1	1	1	7
8	0	0	0	0

Timing diagram. Q0 toggles at every falling clock edge, Q1 toggles when Q0 falls, Q2 toggles when Q1 falls. Each output has **half the frequency** of the one before. The small shifts between the lines are the **ripple delays**.



QUICK TRICK

Up or down? Which output drives the next clock?

Flip-flops react on...	Up counter: next clock from	Down counter: next clock from
Falling edge (negative edge)	Q	Q'
Rising edge (positive edge)	Q'	Q

QUICK TRICK

Why? For counting up, the next flip-flop must toggle when the present bit goes from 1 to 0 (a carry). A falling-edge flip-flop sees that as a falling edge of Q. A rising-edge flip-flop sees it as a **rising** edge of Q'. For a down counter it is the opposite.

Frequency division

Each flip-flop divides the frequency by 2. The output of the last flip-flop of an n-bit counter has frequency $f / 2^n$.

Output	Frequency (clock f)	Example for f = 8 MHz
Q0	$f / 2$	4 MHz
Q1	$f / 4$	2 MHz
Q2	$f / 8$	1 MHz

The problem: ripple delay

The flip-flops do not change together. FF1 changes only after FF0, FF2 after FF1... so the delays **add**. The counter needs time to settle before the next clock pulse.

Settling time = $n \times t_{pd}$ Maximum clock frequency $f_{max} = 1 / (n \times t_{pd})$
(add the delay of any decoding gate if the outputs are decoded)

Example (Level 3)

Given: a 4-bit ripple counter, each flip-flop has $t_{pd} = 20$ ns.

Concept: the last output changes after 4 flip-flop delays.

Calculation: settling time = $4 \times 20 = 80$ ns $\rightarrow f_{max} = 1 / 80$ ns.

Answer: 12.5 MHz.

EXAM TRAP

Ripple counters are slow, and when you decode their outputs (for example to detect a count) the outputs can show wrong values for a very short time (**glitches**).

15.3 Synchronous counter

● LEVEL 2 • EXAM

Idea. Give the **same clock** to all flip-flops, so they change together. To make each flip-flop toggle at the right time, we feed its T input with a signal that is 1 only when it must toggle.

When must a bit toggle in an up counter? A bit toggles when **all lower bits are 1** (because the count carries into it). Look: 0111 $\rightarrow$ 1000. Bit 3 toggles because bits 2, 1, 0 were all 1.

$T_0 = 1$ $T_1 = Q_0$ $T_2 = Q_0 \cdot Q_1$ $T_3 = Q_0 \cdot Q_1 \cdot Q_2$ (up counter)

QUICK TRICK

Reset-gate trick: feed the NAND gate with the outputs that are **1 in the number N**. Mod-6: $N = 110 \rightarrow Q2, Q1$. Mod-10 (decade): $N = 1010 \rightarrow Q3, Q1$. Mod-12: $N = 1100 \rightarrow Q3, Q2$. *Why it works:* any number that has all the 1-bits of N is at least N. So the first time all those bits are 1, the count is exactly N.

Counter	Flip-flops	Count sequence	NAND inputs	Unused states
Mod-6	3	0 to 5	Q2, Q1	2
Mod-10 (decade, BCD)	4	0 to 9	Q3, Q1	6
Mod-12	4	0 to 11	Q3, Q2	4
Mod-5	3	0 to 4	Q2 (and Q0 if you want)	3

EASY MEMORY

Unused states = $2^n - N$. For example a decade counter uses 10 of 16 states, so 6 are unused.

Method 2: synchronous design (flip-flop equations)

Example: a synchronous mod-5 counter that counts 0, 1, 2, 3, 4, 0... using D flip-flops ($D = \text{next state}$). 5 states need 3 flip-flops. The states 5, 6, 7 are unused, so they are **don't cares**.

Present Q2 Q1 Q0	Next Q2 Q1 Q0	D2	D1	D0
000	001	0	0	1
001	010	0	1	0
010	011	0	1	1
011	100	1	0	0
100	000	0	0	0
101	X X X	X	X	X
110	X X X	X	X	X
111	X X X	X	X	X

K-maps (using the X's to make big groups) give:

$$D2 = Q1 \cdot Q0$$

$$D1 = Q1 \text{ xor } Q0$$

$$D0 = Q2' \cdot Q0'$$

QUICK TRICK

Always check the unused states! Using the equations: $101 \rightarrow 010$, $110 \rightarrow 010$, $111 \rightarrow 100$. All of them lead back into the valid counting cycle, so this counter is **self-starting**. (If an unused state led to a loop of unused states the counter would be **locked out**.)

15.5 Ring counter and Johnson counter

● LEVEL 3 • GATE

Both are **shift registers** whose output is fed back to the input (see Chapter 14).

	Ring counter	Johnson (twisted ring) counter
Feedback	Q0 → serial input	Q0' → serial input
Start	1000 (one 1, rest 0) — must be preset	0000
Sequence (4 FFs)	1000 → 0100 → 0010 → 0001 → repeat	0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → repeat
States used	n	2n
Unused states (n = 4)	16 – 4 = 12	16 – 8 = 8
Flip-flops for N states	N	N / 2 (N even)
Decoding a state	None! Each flip-flop output is already a one-hot state	Only a 2-input AND gate per state
Self-starting?	No. Must be initialised.	No. Invalid states form a separate loop (e.g. 0101 → 0010 → 1001 → ...) and need correction logic.

QUICK TRICK

Johnson trick: inverting the feedback doubles the number of states: **n flip-flops → 2n states**. The pattern is: fill with 1s one by one from the left (0000, 1000, 1100, 1110, 1111), then fill with 0s one by one (0111, 0011, 0001, 0000).

GATE POINT

A mod-10 counter can be made as a **5-flip-flop Johnson counter** (10 states), with only 2-input AND gates for decoding. A ring counter for 10 states would need 10 flip-flops.

15.6 Cascading counters and frequency division

● LEVEL 3 • GATE

Join a mod-A counter and a mod-B counter (the output of the first clocks the second). The result is a **mod-(A × B)** counter. The final output frequency is $f / (A \times B)$.

Example: a 60 Hz power line to 1 Hz (a clock)

Mod-6 counter followed by a mod-10 counter → mod-60. $60 \text{ Hz} \div 60 = \mathbf{1 \text{ Hz}}$.

Mod-10 from a decade counter, mod-6 from a 3-flip-flop counter reset at 6. Total flip-flops: $4 + 3 = 7$.

Quick-Solve Tricks — Summary

Topic	Trick
States	$n \text{ flip-flops} = 2^n \text{ states (mod } 2^n)$. Mod-N needs $\lceil \log_2 N \rceil$ flip-flops.
After k pulses	The count is $k \text{ mod } N$.
Ripple clocking	Falling-edge FFs: up = Q, down = Q'. Rising-edge FFs: up = Q', down = Q.
Frequency division	Last output of an n-bit counter = $f / 2^n$. Mod-N = f / N .
Ripple speed	$f_{\text{max}} = 1 / (n \times t_{\text{pd}})$.

Topic	Trick
Synchronous T inputs	Up: $T_i = \text{AND of lower bits}$. Down: AND of complemented lower bits.
Mod-N reset	NAND of the outputs that are 1 in N. Used with asynchronous clear.
Unused states	$2^n - N$. Check that they return to the cycle.
Ring / Johnson	Ring n states, Johnson $2n$ states. Johnson feeds back Q_0' .
Cascading	Mod-A then mod-B = mod-(A × B).

GATE QUICK CHECK — Chapter 15

States = 2^n Flip-flops for mod-N = $\text{ceil}(\log_2 N)$ $f_{\text{out}} = f / N$
 Ripple $f_{\text{max}} = 1 / (n t_{\text{pd}})$ Sync $T_i = Q_0 Q_1 \dots Q_{(i-1)}$ (up)
 Mod-N reset : NAND of the 1-bits of N Ring = n states, Johnson = $2n$ states

Traps: Q or Q' to the next clock (edge type matters) · the reset state N appears for an instant · Johnson has $2n$ states · ripple delay adds up · unused states and lock-out.

Practice questions

● LEVEL 1 · BEGINNER

- Q1.** How many states does a 4-bit counter have?
- Q2.** A 3-bit ripple counter is clocked at 8 MHz. What is the frequency at the last output?
- Q3.** How many flip-flops are needed for a mod-10 counter?

● LEVEL 2 · EXAM

- Q4.** A 4-bit binary counter starts at 0000. What does it show after 37 clock pulses?
- Q5.** For an asynchronous mod-6 counter (3 flip-flops), which outputs go to the reset NAND gate?
- Q6.** A 4-bit ripple counter has flip-flops with $t_{\text{pd}} = 20$ ns. Find the maximum clock frequency.

● LEVEL 3 · GATE

- Q7.** A mod-6 counter is followed by a mod-10 counter. The input frequency is 60 Hz. Find the output frequency and the total modulus.
- Q8.** Synchronous mod-5 counter: $D_2 = Q_1 Q_0$, $D_1 = Q_1 \oplus Q_0$, $D_0 = Q_2' Q_0'$. If it powers up in state 101, what is the next state?
- Q9.** How many flip-flops are needed for a mod-12 Johnson counter? For a mod-12 ring counter?
- Q10.** A 4-bit Johnson counter starts at 0000. Find its value after 5 clock pulses.
- Q11.** Write the T input of the 4th flip-flop (Q_3) in a 4-bit synchronous up counter.
- Q12.** A ripple up counter is built with **positive-edge-triggered** flip-flops. Which output of each flip-flop must drive the clock of the next one?

Answers

Q	Solution
1	$2^4 = \mathbf{16}$ states.
2	$f / 2^3 = 8 \text{ MHz} / 8 = \mathbf{1 \text{ MHz}}$.
3	$\lceil \log_2 10 \rceil = \mathbf{4}$ flip-flops.
4	$37 \bmod 16 = 5 \rightarrow \mathbf{0101}$.

Q	Solution
5	The first count 6 = 110 has $Q_2 = 1$ and $Q_1 = 1$. Use Q2 and Q1 (NAND gate).
6	Given: $n = 4$, $t_{pd} = 20$ ns. Concept: $f_{max} = 1/(n t_{pd})$. Calculation: $1/80$ ns. Answer: 12.5 MHz.
7	Total modulus = $6 \times 10 = 60$. Output = $60 \text{ Hz} / 60 = 1 \text{ Hz}$.
8	$D_2 = 0 \cdot 1 = 0$, $D_1 = 0 \oplus 1 = 1$, $D_0 = 0' \cdot 1' = 1 \cdot 0 = 0 \rightarrow$ next state 010 (a valid state, so the counter recovers).
9	Johnson: $N / 2 = 6$ flip-flops. Ring: 12 flip-flops.
10	$0000 \rightarrow 1000 \rightarrow 1100 \rightarrow 1110 \rightarrow 1111 \rightarrow \mathbf{0111}$.
11	$T_3 = \mathbf{Q_0 \cdot Q_1 \cdot Q_2}$ (all lower bits).
12	A positive-edge flip-flop toggles when its clock rises. The next stage must toggle when the present bit falls ($1 \rightarrow 0$), which is a rising edge of Q' . So use Q' .

Next: Chapter 16 — Memory (organisation, address and data lines, capacity, RAM, SRAM, DRAM, ROM types, memory chip design).